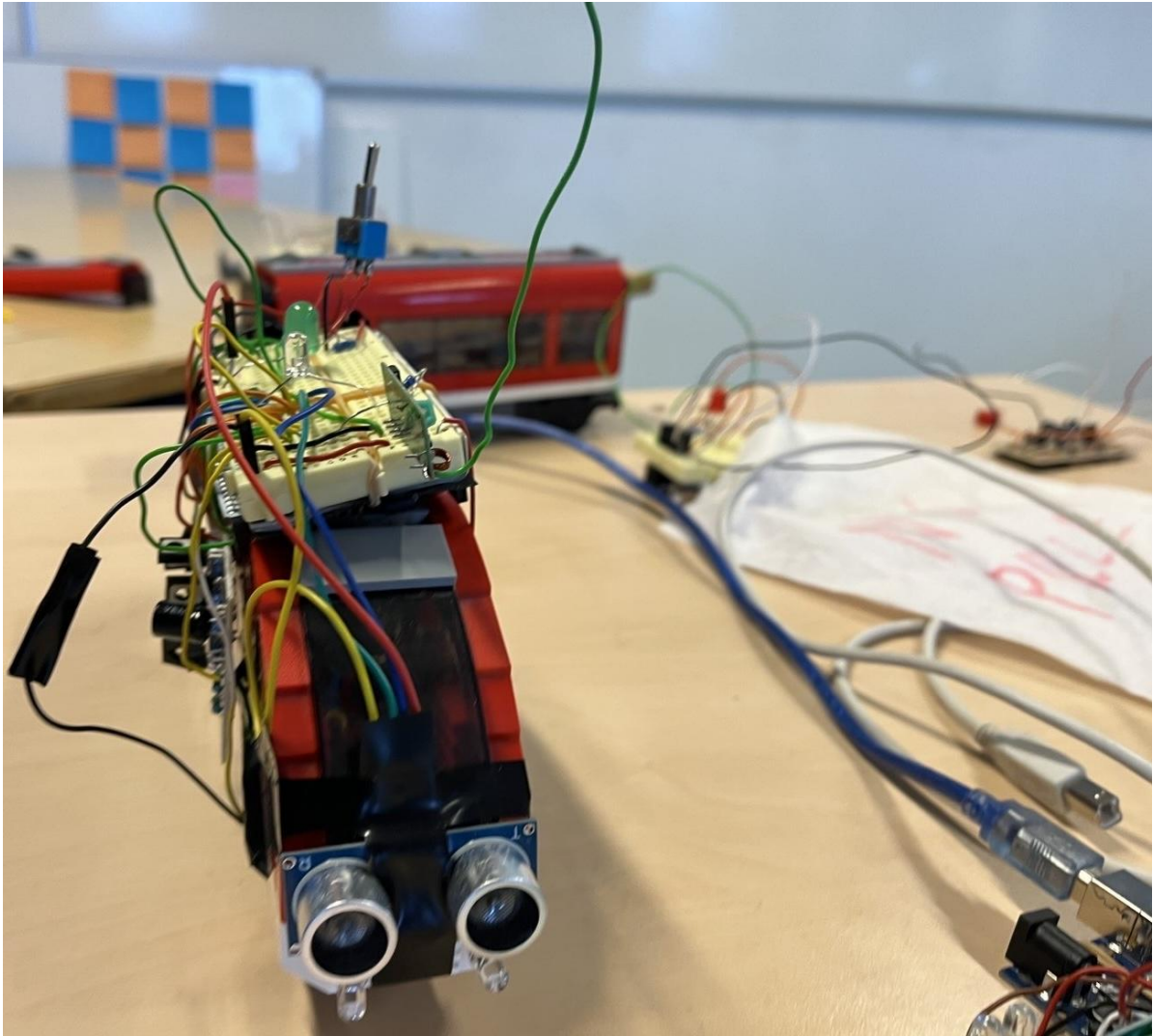


Projekt tog



Skole: HTX Køge

Lærer: Jens Borup

Forfatter: Magnus, Benjamin, Luka og Jaqueline

Aflevering: 10/2-2026

Resume

Projektet handler om udviklingen af et fjernstyret tog med autopilot og sensorer. Formålet er at udvikle et tog, der selv kan køre, stoppe ved stationer og ved blokeringer på skinnerne. Til udformning af dette projekt, blev der først opstillet en række tekniske krav, og herfra undersøgt hvilke komponenter der opfyldte kravene bedst, og gav den bedste oplevelse ved anvendelse af toget. Der blev her undersøgt sensorer med forskellige funktioner, fjernstyring og motorstyring. Projektet inkluderer både praktisk opbygning og programmering i Arduino. Resultatet er et funktionelt tog, der kan køre både manuelt og på autopilot. Toget vurderes ikke til at være helt optimalt ved alle kravene, men som udgangspunkt virker toget, og opfylder de tekniske krav.

Indholdsfortegnelse

Indledning	4
<i>Problemformulering</i>	<i>4</i>
Tekniske krav	4
Ideer og løsningsmuligheder	4
<i>Blokering af skinnerne</i>	<i>5</i>
<i>Fjernstyring.....</i>	<i>6</i>
<i>Forlæns og baglæns.....</i>	<i>7</i>
<i>Manuel kørsel</i>	<i>7</i>
<i>Stations stop.....</i>	<i>8</i>
Tidsplan og budget	9
Produktets virkemåde	11
<i>Kodeudvikling</i>	<i>13</i>
<i>Kredsløbet</i>	<i>15</i>
<i>Udformning af test.....</i>	<i>16</i>
<i>teori og praksis.....</i>	<i>19</i>
Miljøvurdering.....	20
CE-mærkning.....	20
Vurdering.....	21
Konklusion	22
Perspektivering.....	22
Bibliografi	23
Bilagsfortegnelse	24
<i>Bilag 1 – kredsløb over H-bro motorregulering.</i>	<i>24</i>
<i>Bilag 2 – TrainRemote.ino.....</i>	<i>24</i>
<i>Bilag 3 – TrainDriver.ino.....</i>	<i>29</i>

Indledning

Teknologi og automatisering spiller i dag en stor rolle i transportsektoren, hvor der er fokus på både sikkerhed og bæredygtighed. Miljø er i dag et særligt vigtigt emne, og der undersøges hvordan transportmidler kan blive mere grønne, da transportmidler anvendes meget i hverdagen.

I dette projekt arbejdes der med udviklingen af et fjernstyret tog med autopilot og sensorer. Formålet med projektet er at undersøge, hvordan sensorer og styringsteknologi kan anvendes til at gøre tog mere automatiserede og sikre. Projektet kombinerer elektronikarbejde og opbygning med programmering i Arduino for at udvikle et system, der kan styre toget både manuelt og automatisk.

Problemformulering

I denne opgave undersøges muligheden for at fremstille et fjernstyret modeltog med autopilot og afstandscensor i begge ender af toget. Afstandscensorerne observerer om der er noget foran, eller bagved toget, der forhindrer toget i at køre videre. Toget skal selvstændigt kunne køre og stoppe, når det har nået en perron, toget skal også have mulighed for at blive kontrolleret manuelt.

Tekniske krav

Ved udarbejdelse af toget, er der opstillet en række tekniske krav toget skal kunne. De tekniske krav er baseret på at opnå, den bedste oplevelse ved anvendelse af toget.

Kravene lyder følgende:

- Det skal være muligt manuelt at skifte mellem autopilot og manuel kørsel, i form af en fjernbetjening.
- Toget skal have for lygter der manuelt kan tændes og slukkes
- Toget skal detekte om der er en blokering på skinnerne, og kunne stoppe automatisk.
- Toget skal detekte kommende stationer, sænke farten og stoppe automatisk.
- Det skal være muligt at styre hastigheden og retningen af toget manuelt.

Ideer og løsningsmuligheder

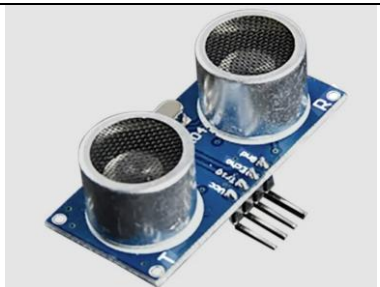
Til at udvikle toget og opfylde de tekniske krav, er der gjort overvejelser om valget af komponenter. Toget har specifikke funktioner der ønskes at få opfyldt, og der undersøges derfor hvilket komponent der giver den bedste løsning.

Blokering af skinnerne

Hvis skinnerne bliver blokeret, skal toget kunne stoppe automatisk når toget kører på autopilot. Til at udføre denne ide er der valgt to sensorer, en ultralydssensor og en bevægelsessensor, der vurderes.

Ultralydssensor

Ultralydscensoren virker ved, at den sender lydbølger ud i rummet og modtager lydbølgerne tilbage hvis der ligger en genstand på skinnerne (Ardustore.dk , 2025).

Udseende	
Pris	20kr.
Fordele	Den kan måle afstand til genstande der står stille og er i bevægelse.
Ulemper	Sender lydbølger ud, som specielt hunde kan høre, der kan have en negativ virkning på dem.

Bevægelsessensor

Bevægelsescensoren virker ved at den registrer bevægelse i sit dækningsområde, ved at måle varme forskelle. På denne måde kan den registrere, hvis noget bevæger sig på skinnerne (Ardustore.dk , 2025).

Udseende	
Pris	20kr.
Fordele	Registrerer hurtigt om noget bevæger sig foran toget.
Ulemper	Måler ikke afstand, kun bevægelse.

	Registrer ikke genstande der står stille.
--	---

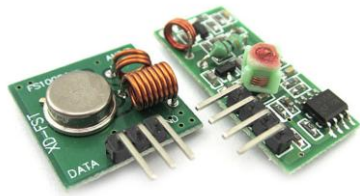
Ideen er at toget skal stoppe, hvis skinnerne bliver blokeret. Skinnerne kan både blive blokeret at genstande i bevægelse og stillestående, som betyder for at opfylde ideen, at ultralydssensoren vælges.

Fjernstyring

Toget skal fjernstyres. Det skal her være muligt at vælge mellem at styre toget manuelt eller på autopilot. Til udvikling af det overvejes en radiofrekvens sender/modtager eller en infrarød modtager.

433MHz Trådløs RF Data Sender/Modtager Module.

Modtageren og senderen fungerer ved, at der placeres en antenne ved hvert modul og de forbinder via 443Mhz signalet. Senderen frigør bølger i rummet, og modtageren opfanger indkommende bølger (Ardustore.dk , 2025).

Udseende	
Pris	25kr.
Fordele	Kan opfange signal fra lang afstand, lavt strømforbrug og simpel i Arduino.
Ulemper	Komponenten er højt anvendt, og der kan være interferens. Lav datahastighed.

IR Infrarød modtager Modul

Senderen udsender infrarødt lys, som modtageren registrerer. For at signalet kan opfanges, skal modtageren være placeret synligt i forhold til senderen, da infrarødt lys kræver fri sigtelinje (Ardustore.dk , 2025).

Udseende	
----------	---

Pris	15kr.
Fordele	Stabilt signal, billig og simpel til Arduino.
Ulemper	Kræver at senderen vender mod modtageren og kort rækkevidde.

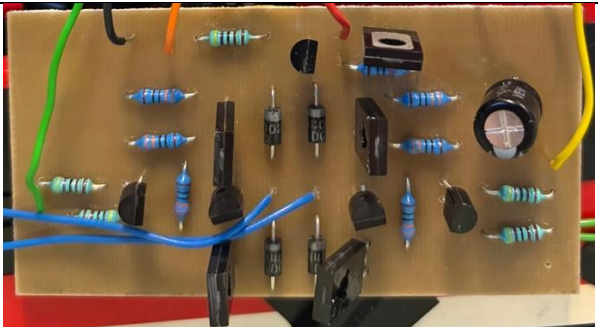
Da toget skal fjernstyres, og herfra bliver sat til at køre rundt, ville oplevelsen blive forværret hvis, "fjernbetjeningen" skulle følge toget. Udover det skal fjernstyringen kunne fungere ved afstand fra toget. Af denne grund vælges 433MHz Trådløs RF Data Sender/Modtager Modul.

Forlæns og baglæns

Toget skal kunne køre forlæns og baglæns. Dette udføres med en H-bro.

H-bro

En H-bro er et elektronisk kredsløb, der gør det muligt at styre retningen og hastighed af en DC-motor. Den fungerer ved at vende polaritet af spændingen i DC-motoren (podconsultsbutik.dk, 2025).

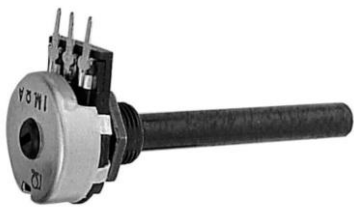
Udseende	
Pris	56,2kr.
Fordele	Skifte retning og hastighedsstyring.
Ulemper	Mister energi som varme, begrænset strøm og indviklet udseende.

Manuel kørsel

Toget skal kunne styre manuelt, og til det skal der vælges en styringsform. Styringsformen som den eneste der er blevet overvejet og dermed valgt, er potentiometret.

Potentiometer

Et potentiometer er en variable modstand, der kan styre strømmen der kommer ud fra outputtet. Den fungerer ved, at når der drejes på potentiometret, skrues der op for modstanden og strømmen kontrolleres (Av-Cables, 2025).

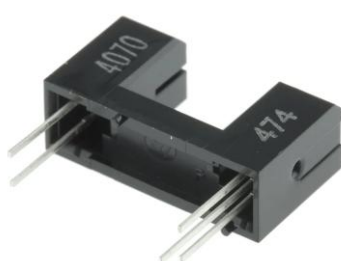
Udseende	
Pris	30kr.
Fordele	Manuel skrue op og ned for strømmen.
Ulemper	Svær til rolig kontakt

Stations stop

Når toget kører på autopilot, skal toget kunne stoppe ved kommende stationer. Til det overvejes en photomicrosensor eller en infrarødsensor.

Photomicrosensor

En photomicrosensor er en optisk sensor, der registrer objekter ved at udsende lys. Hvis der sker en ændring i lysstrålen, vil en modtager detektere ændringerne (Omron, 2025).

Udseende	
Pris	29,36kr.
Fordele	Lille, hurtig respons og lavt strømforbrug
Ulemper	Følsom og kræver korrekt justering.

Infrarødsensor

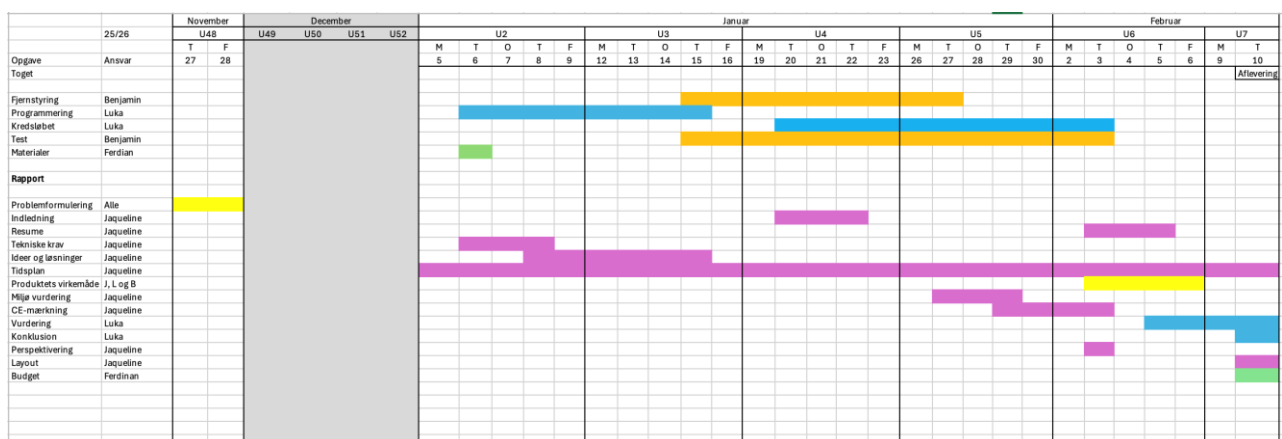
En infrarødsensor virker ved at en LED udsender infrarødt lys, når dette lys, så rammer et specielt materiale ville de blive reflekteret og sensoren ville detektere det (Elextra, 2025).

Udseende	
Pris	34kr.
Fordele	Hurtig respons og lavt strømforbrug
Ulemper	Følsom over for lys og kort rækkevidde.

Til udviklingen af stations stop vælges en photomicrosensor. Begge sensorer har samme fordele der søges til dette tog, men infrarødsensoren er følsom over for lys, som der typisk er på stationer. For at give den bedste oplevelse, vælges derfor photomicrosensor.

Tidsplan og budget

Udarbejdelse af toget og rapporten, er blevet planlagt via en tidsplan. Denne har gjort det muligt, at se hvad der manglede undervejs og hvor vi var i processen, se figur 1.



Figur 1- Tidsplan

Beregning af budget udarbejdet med følgende kilde (EL-SUPPLY, 2025)

H-bro

Pris:

1x modstander 2,65kr. 12 i det hele

1x diode 1,0kr. 4 i det hele

1x 136 2,95kr 2 i det hele

1x 135 2,75kr. 2 i det hele

1x transistor 1,50kr. 6 i det hele

Samlet: 56,2kr

Tog Remote

Reference	Qty	Value
A1	1	Arduino_Leonardo
D1,D2,D3,D4	4	LED
R1,R2,R3,R4,RV1	5	100
RF_433MHz_Transmitter	1	~
SW1	1	SW_DPST_x2

Pris

1x LED 5 kr. 4 i det hele

1x Vippeomskifter 7,5 kr. 2 i det hele

1x Drejepotentiometer 100 ohm 18,25 kr. 1 i det hele

1x Arduino uno R3 229 kr. 1 i det hele

1x Trådløs RF Data Seat købe 100, end kun at købe en 25 kr. 1 i det hele

Samlet: 307,25kr

Tog

Reference	Qty	Value
A1	1	Arduino_UNO_R2
D1,Headlight_1,Headlight_2	4	LED
H-Bro_mortorregulering1	1	H-bro
HC-SR2	1	Scencor
M1	1	Motor_DC
R1,R2,R3,R4,R5	5	100
RF433MHzReciver1	1	Resiver
SW1	1	SW_DPST_x2
U1	1	LG206D

1x LED 5 kr. 4 i det hele

1x Sensor 20 kr. 1 i det hele

1x Resistor 2 kr. 5 i det hele

1x Resiver 25 kr. 1 i det hele

1x Arduino uno R3 229 kr. 1 i det hele

1x Vippeomskifter 7,5 kr. 1 i det hele

1x Photomicrosensor 29,36 kr. 1 i det hele

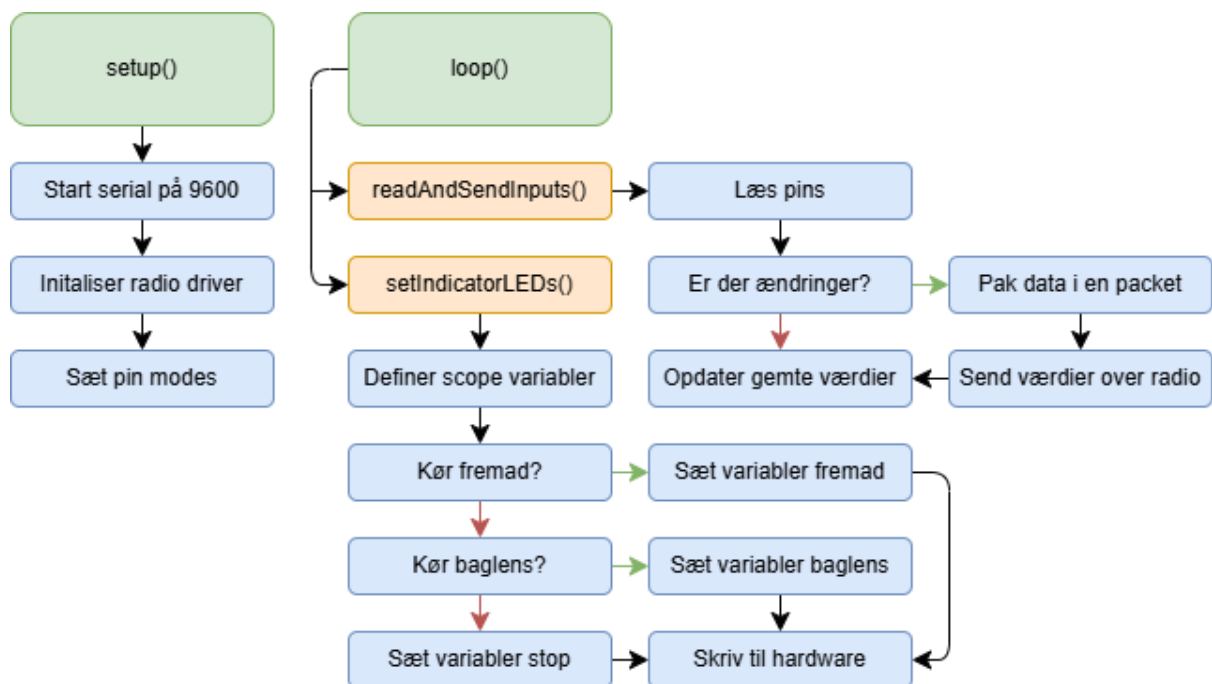
Samlet: 340,86kr.

Togets fulde værdi er 704,31kr.

Ved udregning af budget er der som udgangspunkt, regnet ud fra stykprisen. Når der handles med elektroniske komponenter som modstander, LED og ledninger kan det økonomisk bedre forsvare sig, at købe flere på en gang. I flere tilfælde er det billigere at købe 100, end kun at købe en.

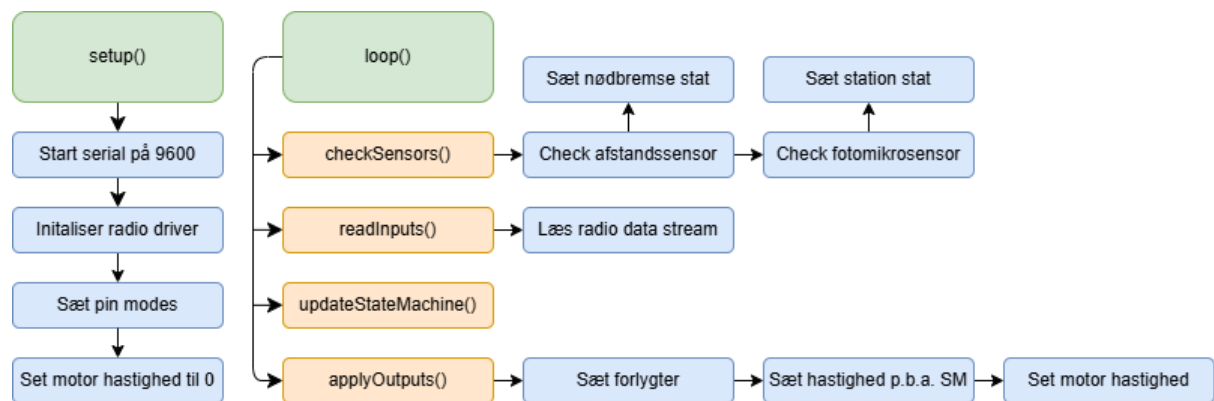
Produktets virkemåde

For at beskrive programmets virkemåde, er der opstillet et blokdiagram, se nedenfor.



Diagrammet ovenstående viser hvordan kode logikken i fjernbetjeningen virker. Hvor de blå det er små kodeblokke - handler inde i en funktion, og de orange er selve funktionen. Det er ikke alle funktioner der er med i diagrammet, kun dem som loop() kalder på. De grønne er funktioner som Arduinoen kalder på selv, setup() er start

funktionen der bliver kaldet på én gang. Loop funktionen bliver kaldet på så oftest CPU'en har mulighed.



Koden ovenstående beskriver togets logik. Toget bruger en *state machine* (SM) til at fungere og finde ud af hvad den skal gøre. En SM tager kontekst om toget så som, om nødsensoren er aktiv, om autopilot er tændt og hvad andre parameter siger. Ud fra det indgår den i et switch statement med under if else statments, hvor den derefter beslutter hvilken ny stat den skal have.

Kodeudvikling

```
1  /**
2  * @brief Read the inputs from the serial and load their values into the global
3  * variables.
4  */
5  void readInputs() {
6      uint8_t buf[15];
7      uint8_t buflen = sizeof(buf);
8      if (rf_driver.recv(buf, &buflen)) {
9          //Serial.print(buf[0], BIN);
10         //Serial.println(buf[1], BIN);
11
12         Packet pkt;
13         decodeStreamByte(buf[0], pkt);
14         if (decodeStreamByte(buf[1], pkt)) {
15             // Packet is fully received, update variables
16             manual_speed = pkt.speed;
17             autopilot_mode = pkt.autopilot;
18             headlights_on = pkt.headlights;
19         }
20     }
21 }
22
23 /**
24 * @brief Decodes the first and second byte from the serial stream, and puts
25 * them into a packet, passed by reference.
26 * @param byte_in The byte from the serial read.
27 * @param out The reference to the packet currently being built.
28 * @return bool, true if the packet is done being built, false if not.
29 */
30 bool decodeStreamByte(uint8_t byte_in, Packet &out) {
31     // Static variables for the next call
32     static uint8_t last_check = 0; // Store an ID, consistency check
33     static uint8_t speed_h = 0; // Hold the top 5 bits of the speed
34     static bool waiting_for_second = false; // Flag, tells if we are mid-packet
35
36     uint8_t check = byte_in >> 6; // Shift to right, isolate check
37     bool frame = (byte_in >> 5) & 0x01; // Shift to right, mask the frame
38
39     // First byte - the setup
40     if (frame == 0) {
41         speed_h = byte_in & 0x1F; // Mask the last 5 bits
42         last_check = check; // Remember the check ID
43         waiting_for_second = true; // Set the flag
44         return false; // Packet is not complete yet
45     }
46
47     // Second byte - but never saw the first
48     if (!waiting_for_second)
49         return false; // Out of sync, discard
50
51     // Check doesn't match
52     if (check != last_check) {
53         waiting_for_second = false; // Lower flag
54         return false; // Check mismatch
55     }
56
57     // Reassemble packet
58     uint8_t speed_l = (byte_in >> 2) & 0x07; // Shift to the right, then mask
59
60     out.speed = (speed_h << 3) | speed_l; // Shift prev to left, then append
61     out.autopilot = (byte_in >> 1) & 0x01; // Shift one to right, then mask
62     out.headlights = byte_in & 0x01; // Mask the last bit
63
64     // Packet is valid
65     waiting_for_second = false;
66     return true;
67 }
```

Ovenstående kode sørger for at sikre god kommunikation mellem transmitter og receiverhardwaren, specifikt er koden her lavet til recieveren som sørger for at modtage to bytes af data lige efter sendt en efter en, og decode dem indtil brugbare data som kan bruges i koden. De data den tager imod er den manuelle hastighed fjernbetjeningen siger toget skal køre, om togets autopilot er til og om headlights er tændt. Den information optager ikke alle 2x8 bits, derfor bliver der sendt check data som sikre at koden ved hvilken byte er den første, og anden, om om dataen der bliver sendt er valid og ikke bare støj.

Koden der sørger for modtagelse, består af to funktioner. I den første funktion checker koden om der er data der venter, hvis der er, så optager den en buffer hukommelse til dataen på 2 bytes, og då prøver den at udfolde data strømmen. Den udfolder strømmen

i den anden funktion, det gør den ved brug af bitshifting, bitmasking, og bitappending som alt sammen er logik der bruges når der arbejdes direkte med binære værdier.

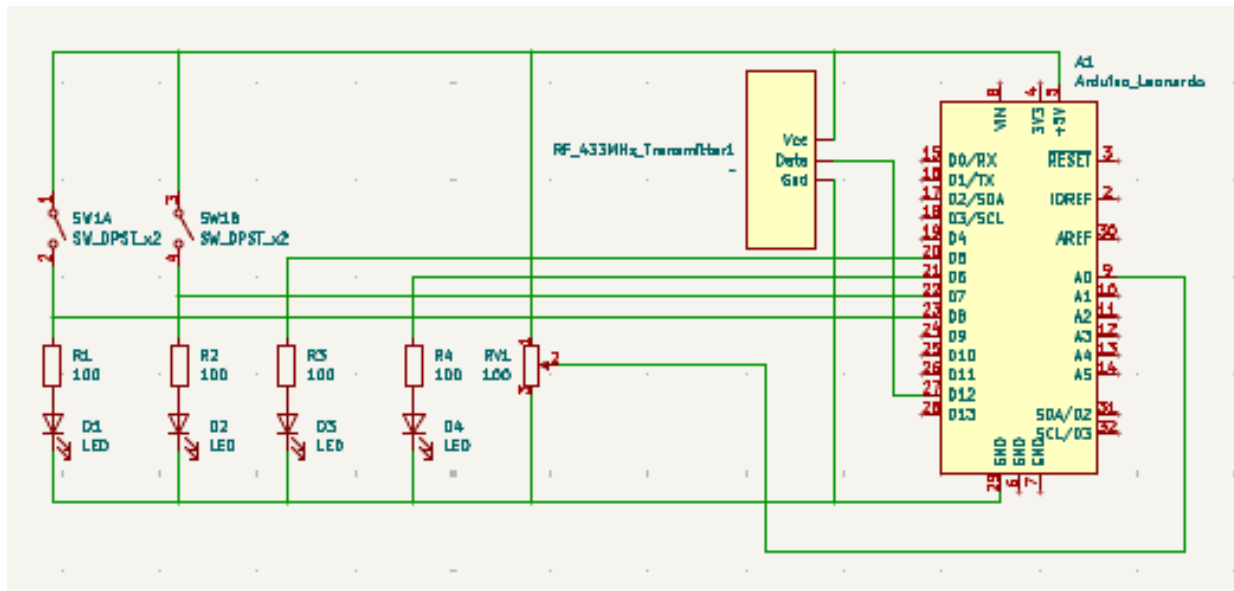
```
1  /**
2  * @brief Read pins and transmit their data if changed since last read.
3  */
4  void readAndSendInputs() {
5      // Read values from pins
6      uint8_t pot_value = (uint8_t)map(analogRead(MOTOR_SPEED), 0, 1023, 0, 255);
7      bool headlights = digitalRead(HEADLIGHT_PIN);
8      bool autopilot = digitalRead(AUTOPILOT_PIN);
9
10     // Check if any values are updated
11     if (
12         pot_value != last_manual_speed
13         || headlights != last_headlight_state
14         || autopilot != last_autopilot_state
15     ) {
16         // Pack the values into 2 bytes
17         uint8_t b1, b2;
18         packPacket(pot_value, autopilot, headlights, b1, b2);
19
20         // Send the data to the controllers buffer
21         uint8_t data[2] = {b1, b2};
22         rf_driver.send(data, 2);
23         rf_driver.waitPacketSent();
24     }
25
26     // Update the values
27     last_manual_speed = pot_value;
28     last_headlight_state = headlights;
29     last_autopilot_state = autopilot;
30 }
31
32 /**
33 * @brief Packs speed, autopilot and headlight states into two bytes. Ready to
34 * send over to train controller.
35 * @param speed The speed the train should run at, only set at manual control.
36 * @param autopilot If true, autopilot will be enabled.
37 * @param headlights If true, headlights will be enabled.
38 * @param b1 Reference to the first byte.
39 * @param b2 Reference to the second byte.
40 */
41 void packPacket(uint8_t speed, bool autopilot, bool headlights, uint8_t &b1, uint8_t &b2) {
42     static uint8_t check = 0; // Static check value, used to control bits
43     check = (check + 1) & 0x03; // Rolling 2-bit check
44
45     // Byte 1
46     b1 = (check << 6) | // Check bits
47         (0 << 5) | // Frame = First byte
48         ((speed >> 3) & 0x1F); // Speed, high bits
49
50     // Byte 2
51     b2 = (check << 6) | // Same check bits
52         (1 << 5) | // Frame = Second byte
53         ((speed & 0x07) << 2) | // Speed, low bits
54         ((autopilot & 1) << 1) | // Autopilot flag
55         (headlights & 1); // Headlights flag
56 }
```

Koden ovenstående består af to funktioner der ligger på fjernbetjeningen til toget, disse funktioner sørger for at sende data over til toget, her vil den data være om headlights er tændt, om autopilot er på, og hvad potentiometerværdien er som repræsenterer den manuelle kontrol hastighed af toget. Koden checker hele tiden hardwareværdierne lokalt, men sender kun data når der sker en ændring, dette sørger for at togets modtager ikke bliver bombet med unødvendigt arbejde.

Når koden antager at der skal sendes en pakke data til toget, skal værdierne pakkes ned i to bytes (2x8 bits) som derefter sendes en efter en. For at sikre toget ved hvilken pakke er den første, og hvilken pakke er den anden, og om den data den modtager overhoved

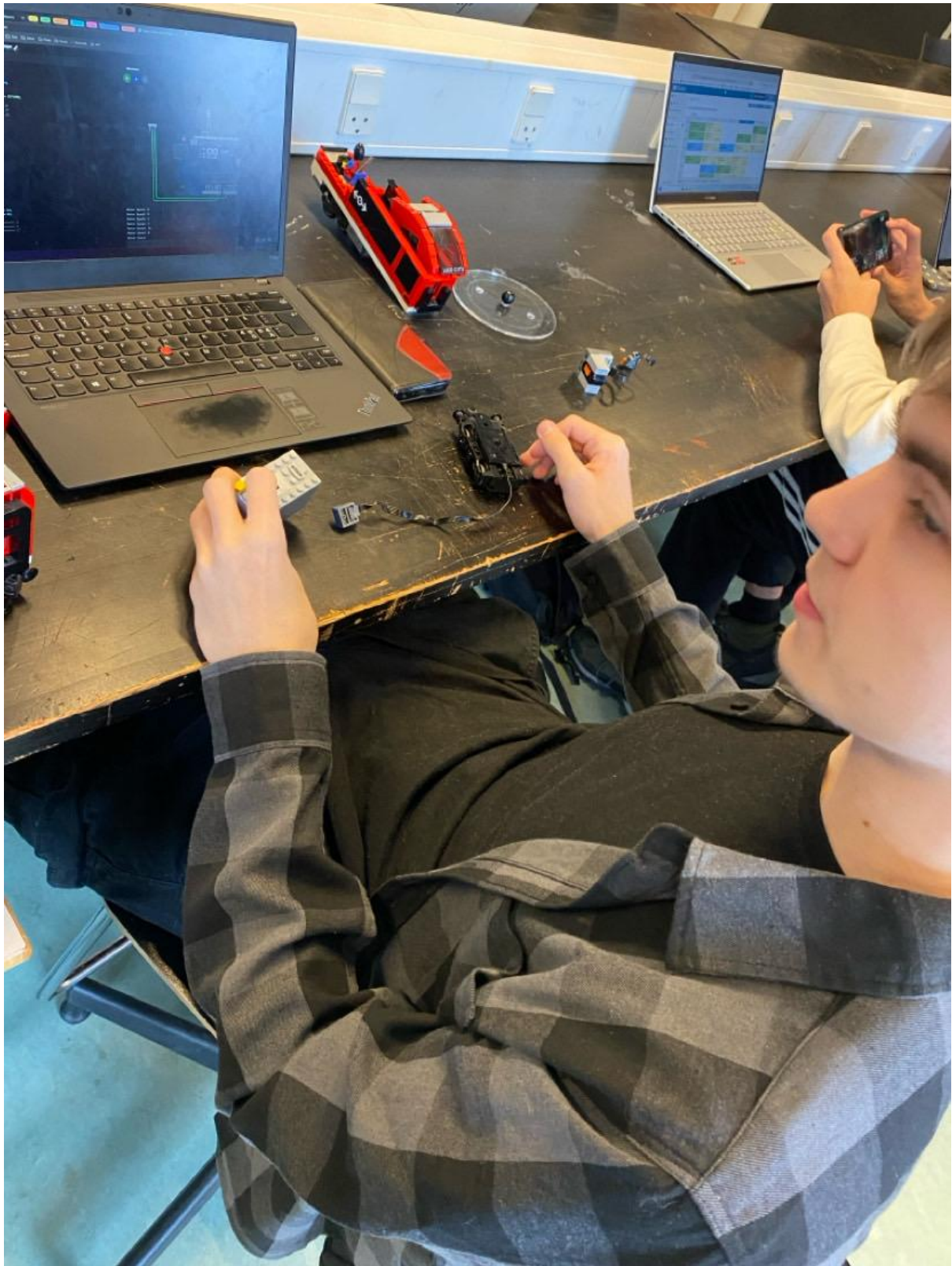
er valid og ikke bare støj, introducere koden check værdier og frame bits. Dette sidder i starten af hver byte og fungere som sikkerhed mod støj og fejl. Dernæst bruges der bitshifting, bitappending og bitmasking for at pakke dataen ned i bytes.

Kredsløbet



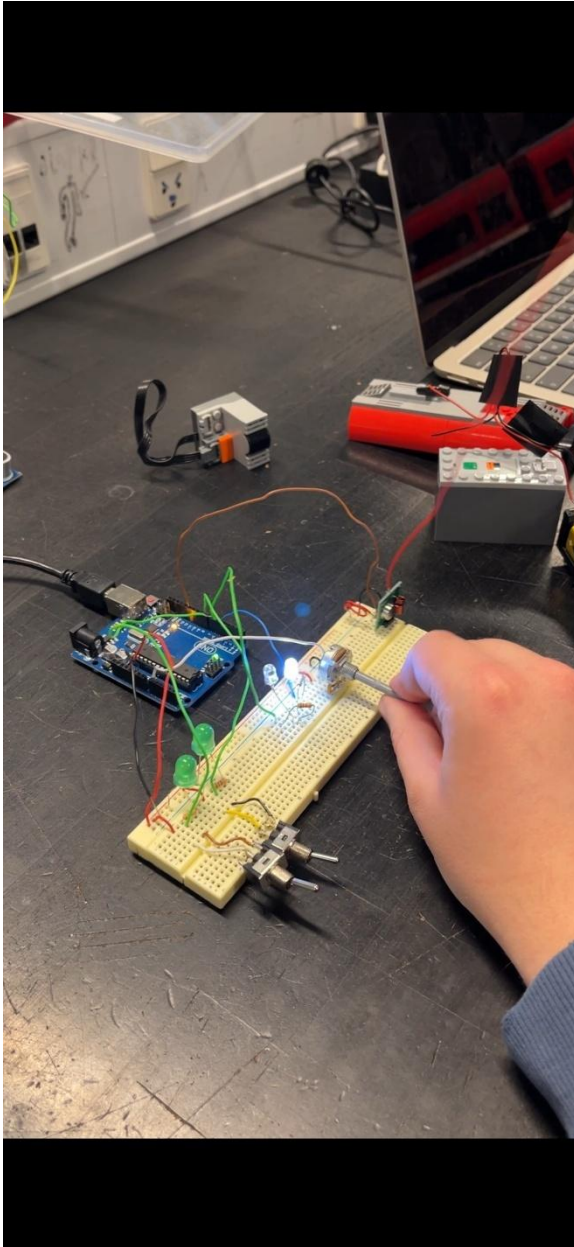
På figuren ses der et kredsløb over tog remoten, kredsløbet består primært af 2 switches, et potentiometer, og RF-transmitter, så består den også af nogle led dioder for at indikere hvilke signaler der bliver sendt. Den ene switch helt til venstre tænder og slukker for headlamps på toget, den anden switch ved siden af tænder og slukker for togets autopilotfunktion, og potentiometeren skruer op og ned for togets hastighed når den styres manuelt. Alt det input bliver sendt ind i Arduinoen som så sender det ud til RF-transmitteren til at sende radio bølger ud til train driver.

Switchende er valgt på grund af at headlamps, og autopilot kun har 2 stager, tændt og slukket, og derfor kun kræver et binært input. Potentiometeret blev valgt på grund af togets hastighed skal variere, og siden et potentiometer er en variabel modstand, kan den bruges til at kontrollere hvor meget spænding der bliver sendt ud.

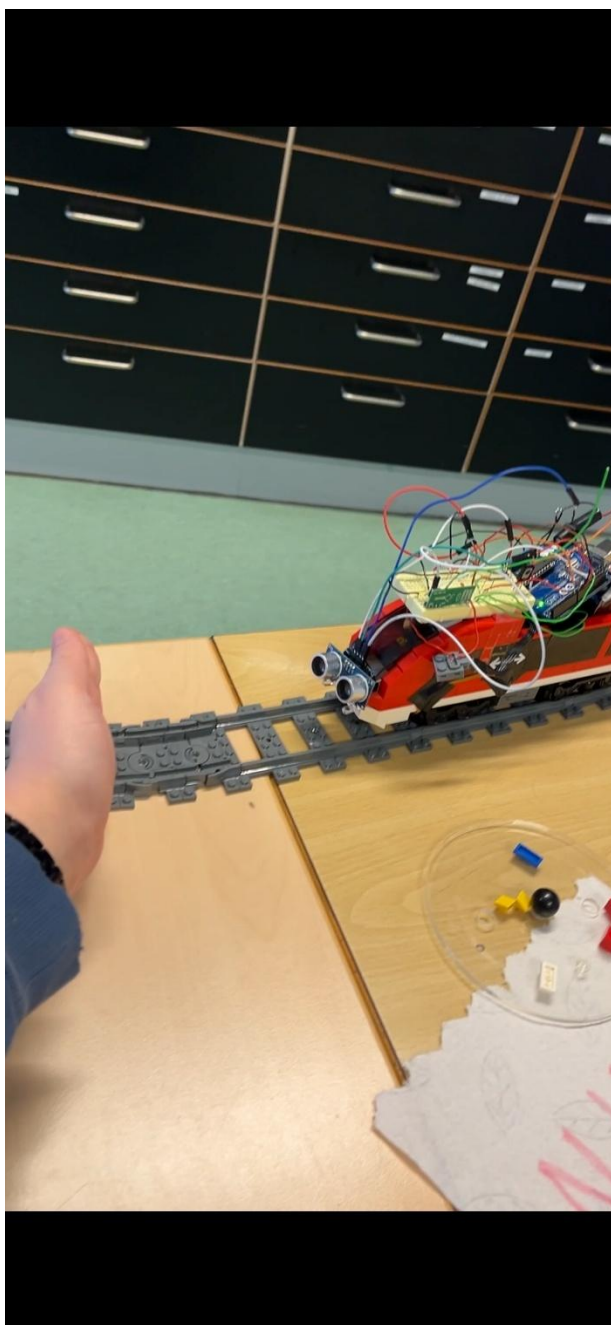


Der blev så fundet frem til at toget har 4 pins, men behøver kun have 2, en for strømforsyning, og en for jord.

Så der bliver der også testet for RF transmitteren og receiveren, her skulle der testes hvordan det var muligt at sende data via radiobølger, hvor der bliver først testet ved at sende en simpel besked der så kunne aflæses på computeren, der det løskedes bliver de implementeret på toget og remoten. Der bliver så nu testet for hvordan man så kunne bruge potentiometer til at sende et output fra remoten til toget som så skal kunne styre hastigheden.



Nu skal der testes for ultralydssensoren, her skal der testes for hvordan den virker og hvor præcis den måler afstand, først bliver der testet for hvordan den virkede ved at koble den op til en Arduino og så måle om den modtog et signal, hvilket den gjorde. Så bliver dens afstandsmåling testet ved at måle dens nøjagtighed med en lineal og se om den modtog et signal når boget var inde for 10 cm, hvilket den gjorde.



Efter det skulle den så emalamateres på toget og nu skulle der testes for om det var muligt at få toget til at stoppe med at køre inde for 20 cm.

Til sidst skal photomicrosensoren testes, først blev den testet med et simpelt kredsløb, der målte når den fik et output. Efter det blev den implanteret på toget hvor der nu skulle testes for om det kunne fungere som en togstations funktion.

teori og praksis

Teorien er brugt i praksis, til at forstå eventuelle tilføjelser af modstandere og undgå kortslutninger. Teorien bruges til at anvende komponenterne korrekt, og få den ønskede

funktion ud af dem. Udover det er teorien også vigtigt, til at arbejde sikkert med kredsløbet i praksis.

Systemets samlede egenskaber udgør opfyldelsen af de tekniske krav. Toget kan køre frem og tilbage, ved brug af en H-bro og toget kan modtage/sende data til fjernstyring, ved brug af en RF sender. Kredsløbet bruges til at tilslutte komponenterne korrekt sammen, og give toget de korrekte funktioner til at opfylde de tekniske krav.

Miljøvurdering

Ved bortskaffelse af toget findes der europæiske regler, som fastsætter korrekt håndtering af elektronisk affald. Dette skyldes, at elektroniske komponenter indeholder forskellige metaller, og nogle kan være giftige ved forkert behandling eller destruktion.

Toget består primært af komponenter, der skal afleveres som elektronikaffald, undtagen batteriet.

Batteriet indeholder miljøfarlige stoffer, som kan forurene jord og grundvand. Derudover kan batterier udgøre en brandfare. Samtidig indeholder batterier materialer, der kan genbruges, hvilket reducerer behovet for ny produktion og dermed skåner miljøet.

CE-mærkning

CE-mærket er adgangsbilletten til det europæiske marked og kræves for at markedsføre og sælge bestemte produkter, herunder elektronik. CE-mærket er producentens erklæring om, at produktet lever op til gældende direktiver og standardernes krav. Det indebærer, at producenten overholder krav til sikkerhed, sundhed og miljø.

Afhængigt af hvilken type elektronik der søges mærkning til, gælder forskellige direktiver. Det kan for eksempel være lavspændingsdirektivet, EMC-direktivet (elektromagnetisk kompatibilitet) og Radio Equipment Directive. For at opnå CE-mærkning kræves dokumentation i form af rapporter, undersøgelser og test, der viser, at produktet overholder kravene.

Specielt overvejende komponent i dette tog er RF-data sender/modtageren, der anvendes til fjernstyring. Denne type komponent udsender radiosignaler og kan skabe elektromagnetisk interferens, hvilket er relevant i forhold til EMC-direktivet. EMC-kravene betyder, at et produkt hverken må forstyrre andre elektroniske enheder eller selv blive forstyrret så det ikke fungerer korrekt.

Motor og motorstyring er også relevante i forhold til EMC-direktivet. DC-motorer udsender elektrisk støj, som kan skabe interferens. Denne interferens kan påvirke andre dele af systemet og bør derfor tages i betragtning.

Toget anvender desuden sensorer og en Arduino-mikrocontroller, som er følsomme over for elektrisk støj. Dette kan forstyrre afstandsmålinger og give fejl i dataoverførslen, hvilket kan reducere systemets stabilitet og funktion. Dette er derfor også relevante forhold at overveje i forbindelse med CE-mærkning.

Vurdering

Den Arduino uno der bruges til projektet kører på en 16MHz clock speed på CPU'en. Dette er et lille problem fordi den også er single thread, hvilket vil sige at den ikke kan udføre to stykker kode på samme tid. Problemet er tydeligt når den skal checke sensore, især sensore som tager tid at hente værdier fra - som ultralydssensor der tager >15ms. Imens den checker denne sensor stopper alt andet kode med at køre, så den kan ikke checke om den er foran en station, den kan ikke gøre toget langsommere, den kan ikke tage imod radiosignaler. Et eksempel på et problem der er aktuelt for toget, er at fotodioedesensoren under toget der holder øje med om der kommer en station den skal stoppe til, nu ikke checker hurtigt nok - så den risikerer at køre forbi en station. En løsning til dette er at begrænse hastigheden for toget så den ikke bare suser forbi en stations-pind som skal opfanges af sensoren under toget.

Med alle funktioner som toget skal kunne udføre, taget i betragtning så kunne selvstyringen af toget godt køre noget bedre. Men når det bliver taget i overvejelse at hele toget og alt den skal kunne er styret af en Arduino Uno med en enorm langsom processor og kun én core, så der ikke meget at gøre. Hvis der skulle optimeres på toget, skulle nogle hardware features og funktioner fjernes - og koden skulle optimeres drastisk.

Karosseriet på toget har også været svært at arbejde med, da toget er bygget for småt til at bære på printplader indvendigt, måtte alt elektronik der styre toget sættes udenpå. Med alt batteri og elektronik var toget overbelastet, men motoren havde ikke noget problem i at trække igennem med få ekstra volt end de standard 6.7V og en optimerede H-Bro.

Sensoren under toget der står for at checke om toget nærmer sig en station eller er nået til slutningen af en perron er ikke optimal i forhold til pålidelighed. Især når den når måden fysiske del som den skal detekte ikke er et objekt egnet til formålet - i projektet blev der brugt elektrikkertape rullet op og skåret ud, som derefter blev stoppet mellem skinnerne. Kombineret med den langsomme CPU på toget, misser den nogle gange stationer når den kører hurtigt, eller skinneindikatoren ikke falder præcis imellem fotosensoren.

Konklusion

Formålet med dette projekt var at udvikle et fjernstyret tog, der kan køre manuelt eller på autopilot med sensorer. Ved autopilot skal toget kunne stoppe ved stationer, og stoppe hvis skinnerne er blokeret.

I dette projekt er det formålet at designe og bygge et fungerende system med Arduino, motorstyring og sensorer. Toget opfylder de tekniske krav vedrørende styringsteknologien. Toget kan med fjernstyring, både styres manuelt, og her vil sensorerne slå fra, og det kan styres på autopilot, hvor sensorerne vil slå til. Ultralydssensoren kan registrere forhindringer og få toget til at stoppe, og photomicrosensor kan registrere stationer, så toget sænker farten og stopper. H-broen gør det muligt at styre retning og hastighed, hvilket var en vigtig del af kravene.

Den manuelle kontrol fungerer ved brug af en RF-forbindelse, hvor den ønskede hastighed, lys og skift mellem manuel og autopilot kan kontrolleres på afstand. Dette sørger for god fleksibel betjening ved afstand, og god brugervenlighed.

Selvom bygrænserne vedrørende Arduino Uno's CPU-hastighed og sensorens pålidelighed ved høj hastighed, fungerer systemet samlet set okay. De tekniske krav er opfyldt, og projektet demonstrer succesfuldt hvordan automatisering, sensorteknologi og trådløs kommunikation kan kombineres i et fungerende transportsystem.

Der kan konkluderes at problemformuleringen er besvaret, og projektets mål om at få udviklet et fjernstyret tog er opnået.

Perspektivering

Ideerne og udviklingen af dette produkt kan være relevante for hvordan fremtidige toge kører og styres, samt gøre det lettere i branchen, kræve færre mennesker og mindske antallet af togulykker. Toget kører på et genopladeligt batteri, der gør det til et el-tog. Det betyder at toget passer ind i den udvikling vi ser i samfundet i dag, med overgangen fra fossile brændstoffer til el. Udover det kan toget køre på autopilot, som vil gøre styring af tog lettere. Toget har automatisk standsning, i tilfælde af at skinnerne er blokeret, og opfanger også bevægelse på skinnerne. Dette bidrager også til at formindske chancen for eventuelle uheld. Dette produkt kan danne et grundlag eller bidrage til teknologiudviklingen inden for nye tog.

Bibliografi

Ardustore.dk . (2025). *433MHz Trådløs RF Data Sender/Modtager Module*. Hentet fra Ardustore.dk : https://ardustore.dk/produkt/433mhz-traadloes-rf-data-sender-modtager-module?gad_source=1&gad_campaignid=21613294775&gbraid=0AAAAAo4di3JGfftsfUHGfYk3LYY9o9BC&gclid=CjwKCAiAs4HMBhBJEiwACrfNZYKwBwLTNq2A5wABKt-Lyq9fDpyxcMDedmwS6vPcHlwKowKrT_cExoClwQQAvD_Bw

Ardustore.dk . (2025). *HC-SR04 Ultrasonic Distance Module*. Hentet fra Ardustore.dk : https://ardustore.dk/produkt/hc-sr04-module?gad_source=1&gad_campaignid=21613294775&gbraid=0AAAAAo4di3J5YhuhgrulGUrVwUekl914f&gclid=Cj0KCQiAy6vMBhDCARIsAK8rOgkCRUy_kjO6mz61g0iUZbTU-dobPy0Vd_cYHhbTmbOQ-P8O4gl4z3YaAu2eEALw_wcB

Ardustore.dk . (2025). *HC-SR505 PIR Bevægelsesføler Module*. Hentet fra Ardustore.dk : https://ardustore.dk/produkt/hc-sr505-pir-bevaegelsesfoeler-module?gad_source=1&gad_campaignid=21613294775&gbraid=0AAAAAo4di3J5YhuhgrulGUrVwUekl914f&gclid=Cj0KCQiAy6vMBhDCARIsAK8rOgknVQFJL5RRmQLeKgoKHZGohwx5TW6lxYlNlo17kSY6pkbiGuRNj50aAjsmEALw_wcB

Ardustore.dk . (2025). *KY-022 Universal IR Infrared Receiver Module*. Hentet fra Ardustore.dk : https://ardustore.dk/produkt/ky-022-universal-ir-infrared-receiver-module?gad_source=1&gad_campaignid=21613294775&gbraid=0AAAAAo4di3JGfftsfUHGfYk3LYY9o9BC&gclid=CjwKCAiA1obMBhAbEiwAsUBbltoO0d8TOk7rJBn7Q5wxDkODJYoS9gaD4aXM4hvOju92xka10eQYVRoCXmYQAvD_BwE

Av-Cables. (2025). *Panelpotentiometer - 100 kΩ*. Hentet fra Av-Cables: https://www.av-cables.dk/adaptore-bokse-og-stik/loese-stik/kontakt/panelpotentiometer---100-k?utm_source=google&utm_medium=cpc&utm_campaign=DK%20%7C%20Standard%20Shopping%20%7C%20Low%20GP%20%7C%2010%20-%2050&utm_id=21940562313&gad_source=1&gad_campaignid=

Elextra. (2025). *Infrarød afstandssensor til Arduino*. Hentet fra Elextra: <https://www.elextra.dk/p/infrarød-afstandssensor-til-arduino-20-300mm-35-33-5v/H43350>

EL-SUPPLY. (2025). *EL-SUPPLY*. Hentet fra EL-SUPPLY: <https://el-supply.dk>

Omron. (2025). *Photomicrosensor*. Hentet fra Omron:

https://omronfs.omron.com/en_US/ecb/products/pdf/en-ee_sx3070_4070.pdf

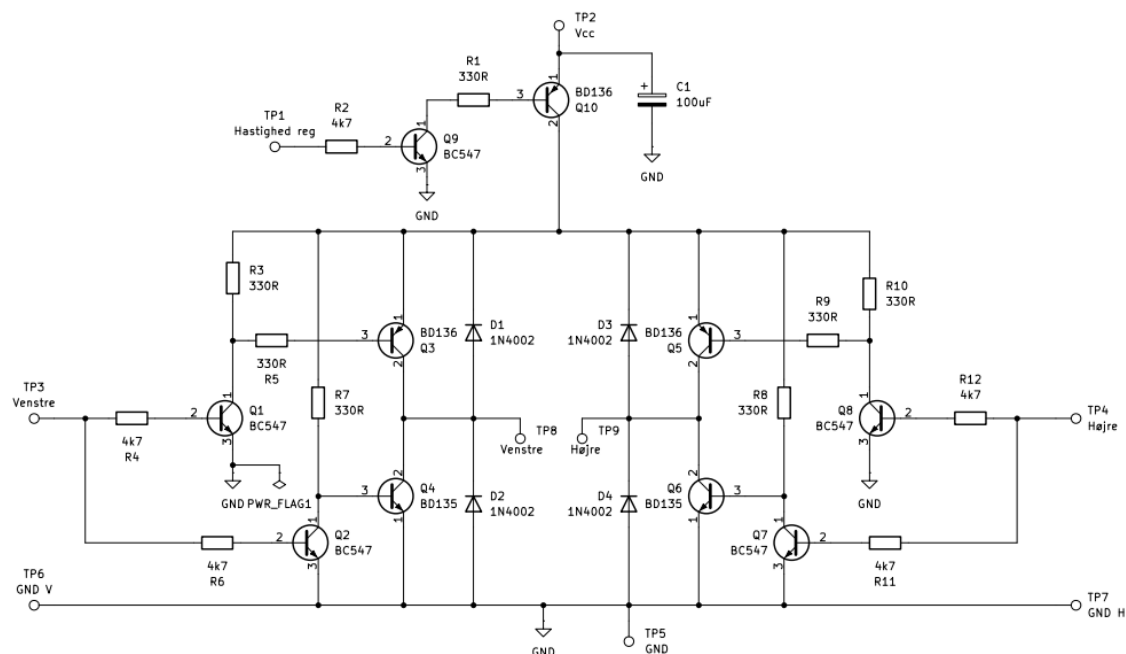
podconsultsbutik.dk. (2025). *H-bro til motorer og micro:bit*. Hentet fra

podconsultsbutik.dk: [https://www.podconsultsbutik.dk/h-bro-til-motorer-og-micro-](https://www.podconsultsbutik.dk/h-bro-til-motorer-og-micro-bit#:~:text=En%20H%2Dbro%20er%20et,at%20køre%20fremad%20eller%20bagud.)

[bit#:~:text=En%20H%2Dbro%20er%20et,at%20køre%20fremad%20eller%20bagud.](https://www.podconsultsbutik.dk/h-bro-til-motorer-og-micro-bit#:~:text=En%20H%2Dbro%20er%20et,at%20køre%20fremad%20eller%20bagud.)

Bilagsfortegnelse

Bilag 1 – kredsløb over H-bro motorregulering.



Bilag 2 – TrainRemote.ino

Link til kode: <https://github.com/LukaRic7/Electric-Model-Train/>

```
/**
 * @file    TrainRemote.ino
 * @author  Luka Jacobsen (Prototyped by Jacq & Benjamin)
 * @brief   Remote control for LEGO model train.
 * @date    2026-01-09
 * * @details This file sends wireless packets to the controller that handles
 * the train. The remote has support for autopilot, headlights and manual
 * speed.
 * * HARDWARECONNECTIONS:
 * - Pin 12: Transmit Wireless (Digital, TX)
 * - Pin 7:  Headlights          (Digital)
```



```

* - Pin 8:  Autopilot      (Digital)
* - Pin A0: Motor speed   (Analog, PWM)
* - Pin 5:  LED Forward Indi (Digital, PWM)
* - Pin 6:  LED Backward Indi (Digital, PWM)
*/

// Include libraries to transmit data
#include <RH_ASK.h>
#include <SPI.h>

// Create transmitter driver
RH_ASK rf_driver;

// =====
//
//
CONFIGURATION //
// =====
//

// -- Transmitter / Input Reading -- //
const int READ_TRANSMIT_DELAY = 20; // (ms) Read and send once every 100ms

// ----- Speeds (0-255) ----- //
const uint8_t DEADZONE = 10; // +- center

// =====
//
// PIN
DEFINITIONS //
// =====
//

const uint8_t MOTOR_SPEED = A0;
const uint8_t HEADLIGHT_PIN = 7;
const uint8_t AUTOPILOT_PIN = 8;
const uint8_t TRANSMIT_SEND = 1;
const uint8_t LED_FORWARD = 5;
const uint8_t LED_BACKWARD = 6;

// =====
//
// RUNTIME GLOBAL
VARIABLES //
// =====
//

// ----- Last read values ----- //

```

```

uint8_t last_manual_speed    = 0;
uint8_t last_headlight_state = false;
uint8_t last_autopilot_state = false;

// =====
//
// LIFECYCLE
FUNCTIONS                                     //
// =====
//

/**
 * @brief Called once on startup by system.
 */
void setup() {
    // Start serial (UART) on baudrate 9600
    Serial.begin(9600);

    // Initialize transmitter driver
    rf_driver.init();

    // Set controller pin modes
    pinMode(MOTOR_SPEED, INPUT_PULLUP);
    pinMode(HEADLIGHT_PIN, INPUT_PULLUP);
    pinMode(AUTOPILOT_PIN, INPUT_PULLUP);

    // Set indicator pin modes
    pinMode(LED_FORWARD, OUTPUT);
    pinMode(LED_BACKWARD, OUTPUT);

    // Set transmitter send pin
    pinMode(TRANSMIT_SEND, OUTPUT);
}

/**
 * @brief Called every time the processor can.
 */
void loop() {
    readAndSendInputs();
    setIndicatorLEDs(last_manual_speed);

    // Debug log
    Serial.println(
        "Manual Speed: "      + String(last_manual_speed)
        + " | Headlight State: " + String(last_headlight_state)
        + " | Autopilot State: " + String(last_autopilot_state)
    );
}

```

```

// =====
//
// Main
Functions                                                                    //
// =====
//

/**
 * @brief Read pins and transmit their data if changed since last read.
 */
void readAndSendInputs() {
    // Read values from pins
    uint8_t pot_value = (uint8_t)map(analogRead(MOTOR_SPEED), 0, 1023, 0, 255);
    bool headlights   = digitalRead(HEADLIGHT_PIN);
    bool autopilot     = digitalRead(AUTOPILOT_PIN);

    // Check if any values are updated
    if (
        pot_value      != last_manual_speed
        || headlights   != last_headlight_state
        || autopilot    != last_autopilot_state
    ) {
        // Pack the values into 2 bytes
        uint8_t b1, b2;
        packPacket(pot_value, autopilot, headlights, b1, b2);

        // Send the data to the controllers buffer
        uint8_t data[2] = {b1, b2};
        rf_driver.send(data, 2);
        rf_driver.waitPacketSent();
    }

    // Update the values
    last_manual_speed      = pot_value;
    last_headlight_state   = headlights;
    last_autopilot_state   = autopilot;
}

// =====
//
// Helper
Functions                                                                    //
// =====
//

/**
 * @brief Packs speed, autopilot and headlight states into two bytes. Ready to

```

```

* send over to train controller.
* @param speed The speed the train should run at, only sat at manual control.
* @param autopilot If true, autopilot will be enabled.
* @param headlights If true, headlights will be enabled.
* @param b1 Reference to the first byte.
* @param b2 Reference to the second byte.
*/
void packPacket(uint8_t speed, bool autopilot, bool headlights, uint8_t &b1,
uint8_t &b2) {
    static uint8_t check = 0;    // Static check value, used to control bits
    check = (check + 1) & 0x03;  // Rolling 2-bit check

    // Byte 1
    b1 = (check << 6) |          // Check bits
        (0 << 5) |              // Frame = First byte
        ((speed >> 3) & 0x1F);  // Speed, high bits

    // Byte 2
    b2 = (check << 6) |          // Same check bits
        (1 << 5) |              // Frame = Second byte
        ((speed & 0x07) << 2) | // Speed, low bits
        ((autopilot & 1) << 1) | // Autopilot flag
        (headlights & 1);       // Headlights flag
}

/**
* @brief Lights up indicators on the remote to give feedback on which
* direction the user is attempting to drive.
* @param speed The speed the train should run at. i.e indicators.
*/
void setIndicatorLEDs(uint8_t speed) {
    // Define scope wide variables
    uint8_t pwm = 0;    // Pulse-Width-Modulation, actual speed to set
    bool forward = false; // Flag for driving forwards
    bool backward = false; // Flag for driving backwards

    // Define thresholds (deadzones) around the center (127)
    int upper_threshold = 127 + DEADZONE;
    int lower_threshold = 127 - DEADZONE;

    // Move forwards
    if (speed > upper_threshold) {
        // Map the speed from threshold
        pwm = (uint8_t)map(speed, upper_threshold, 255, 0, 255);
        forward = true;
    }
    // Move backwards
    else if (speed < lower_threshold) {
        // Map the speed from threshold

```

```

    pwm = (uint8_t)map(lower_threshold - speed, 0, lower_threshold, 0, 255);
    backward = true;
} else {
    // Deadzone: Input is between 117 and 137
    pwm      = 0;
    forward  = false;
    backward = false;
}

// Write to hardware
analogWrite(LED_FORWARD, forward ? pwm : LOW);
analogWrite(LED_BACKWARD, backward ? pwm : LOW);
}

```

Bilag 3 – TrainDriver.ino

```

/**
 * @file    TrainDriver.ino
 * @author  Luka Jacobsen
 * @brief   Bidirectional motor control system for a LEGO model train.
 * @date    2026-01-22
 * * @details This file handles wireless packet decoding, motor ramping, and
an
 * autopilot state machine, aswell as manual control and headlights. Designed
 * for an H-Bridge driver.
 * * HARDWARECONNECTIONS:
 * - Pin 8:  Motor Forward      (Digital)
 * - Pin 7:  Motor Backward     (Digital)
 * - Pin 11: Transmitter        (Digital)
 * - Pin 6:   Motor Speed       (Digital, PWM)
 * - Pin 13: Headlights         (Digital)
 * - Pin 9:   Sensor Trigger    (Digital, PWM)
 * - Pin 10: Sensor Echo        (Digital, PWM)
 * - Pin 4:   Station Sensor    (Digital)
 * - Pin 3:   Autopilot indi    (Digital)
 */

// Include libraries to receive data
#include <RH_ASK.h>
#include <SPI.h>

// Create receiver driver
RH_ASK rf_driver;

//
===== //

```

```

//
CONFIGURATION //
//
===== //

// ----- Speeds (0-255) ----- //
const uint8_t SPEED_CRUISE = 30; // Full forwards
const uint8_t SPEED_SLOW   = 100; // 25% power forwards
const uint8_t ACCEL_RATE   = 40; // +40 pr second
const uint8_t DECEL_RATE   = 60; // -60 pr second
const uint8_t DECEL_RATE_E = 120; // -120 pr second when emergency stopped
const uint8_t DEADZONE     = 10; // +- center

// ----- Timings (ms) ----- //
const unsigned int STATION_DWELL_TIME = 5*1000; // (ms) 2 bytes, max 65 sec

// ----- Emergency (cm) ----- //
const uint8_t EMERGENCY_MARGIN_STOP = 20; // 40cm or closer, stop
const uint8_t EMERGENCY_MARGIN_RETURN = 30; // 60cm away, cancel emergency

//
===== //
// PIN
DEFINITIONS //
//
===== //

// ----- Motor Drivers ----- //
const uint8_t MOTOR_FORWARD = 8;
const uint8_t MOTOR_BACKWARD = 7;
const uint8_t MOTOR_SPEED = 6;

// ----- Indicators ----- //
const uint8_t AUTOPILOT_INDI = 3;

// ----- Headlights ----- //
const uint8_t HEADLIGHT_PIN = 13;

// ----- Sensors ----- //
const uint8_t SENSOR_TRIGGER = 9;
const uint8_t SENSOR_ECHO = 10;
const uint8_t STATION_SENSOR = 4;

//
===== //
// ENUMS /
STRUCTS //

```

```

//
===== //

/**
 * @brief Operating modes of the train, used by the state machine.
 */
enum TrainState : uint8_t {
    STOPPED,           // Zero power, train stopped
    MANUAL_CONTROL,    // Direct control via remote
    AUTO_CRUISING,      // Autopilot constant speed
    AUTO_SLOWING,       // Autopilot slowing for station approach
    EMERGENCY_STOPPED  // Something blocked the front sensor
};

/**
 * @brief Container for wireless control data. Sent from the remote to the
 * train every import change.
 */
struct Packet {
    uint8_t speed;      // Mapped pot speed (0-255)
    bool autopilot;     // True = Autopilot enabled
    bool headlights;    // True = LED Headlights enabled
};

//
===== //

// RUNTIME GLOBAL
VARIABLES                                     //
//
===== //

// ----- Train States ----- //
TrainState train_state      = STOPPED;      // Current state of the train
bool autopilot_mode         = false;        // True = Autopilot enabled
bool headlights_on          = false;        // True = LED Headlights
enabled
unsigned int station_dwell_timer = 0;        // (ms) Timer for station
dwell
bool in_emergency_stop      = false;        // True = Emergency stopped
uint8_t station_trigger_count = 0;          // Counts triggers (0, 1, 2)
bool last_station_sensor    = false;        // Edge detection

// ----- Speeds ----- //
uint8_t manual_speed = 0;    // Manual speed received from remote control
uint8_t target_speed = 0;    // Target speed train is moving towards
uint8_t train_speed  = 0;    // The actual train speed

```



```

//
===== //
// LIFECYCLE
FUNCTIONS //
//
===== //

/**
 * @brief Called once on startup by system.
 */
void setup() {
    // Start serial (UART) on baudrate 9600
    Serial.begin(9600);

    // Initialize ASK driver
    rf_driver.init();

    // Set indicator pin
    pinMode(AUTOPILOT_IND1, OUTPUT);

    // Set motor pin modes
    pinMode(MOTOR_FORWARD, OUTPUT);
    pinMode(MOTOR_BACKWARD, OUTPUT);
    pinMode(MOTOR_SPEED, OUTPUT);

    // Set headlights pin mode
    pinMode(HEADLIGHT_PIN, OUTPUT);

    // Set the sensor pins
    pinMode(SENSOR_TRIGGER, OUTPUT);
    pinMode(SENSOR_ECHO, INPUT);
    pinMode(STATION_SENSOR, INPUT_PULLUP);

    // Set the initial motor speed to 0
    setMotor(0);
}

/**
 * @brief Called every time the processor can.
 */
void loop() {
    // Call main functions, keep loop() clean
    checkSensors();
    readInputs();
    updateStateMachine();
    applyOutputs();

    // Debug log

```

```

Serial.println(
    "Target Speed: " + String(target_speed)
  + " | Train Speed: " + String(train_speed)
  + " | Driving: " + (train_speed == 0 ? "Stopped"
    : (train_speed > 127 ? "Forwards" : "Backwards"))
  + " | Autopilot: " + (autopilot_mode ? "ON" : "OFF")
  + " | Headlights: " + (headlights_on ? "ON" : "OFF")
  + " | Emergency: " + (in_emergency_stop ? "ON" : "OFF")
  + " | Station: " + (last_station_sensor ? "ON" : "OFF")
  + " | Dwell: " + String(millis() - station_dwell_timer)
  + " | State: " + String(train_state)
);
}

//
===== //
// Main
Functions //
//
===== //

/**
 * @brief Check if the train should perform an emergency stop
 */
void checkSensors() {
    // Clear trigger
    digitalWrite(SENSOR_TRIGGER, LOW);
    delay(2);

    // Fire a pulse for 10 ms
    digitalWrite(SENSOR_TRIGGER, HIGH);
    delay(10);
    digitalWrite(SENSOR_TRIGGER, LOW);

    // Read the echo, and calculate the distance
    unsigned long duration = pulseIn(SENSOR_ECHO, HIGH);
    unsigned int distance = duration * 0.034 / 2; // Speed of sound, travel 2
way

    // Check if emergency flag should raise or lower
    if (distance < EMERGENCY_MARGIN_STOP)
        in_emergency_stop = true;
    else if (in_emergency_stop && distance > EMERGENCY_MARGIN_RETURN)
        in_emergency_stop = false;

    // Read station sensor
    bool current_station = digitalRead(STATION_SENSOR);

```

```

    // Rise edge detection
    if (current_station && !last_station_sensor)
        station_trigger_count++;

    last_station_sensor = current_station;
}

/**
 * @brief Read the inputs from the serial and load their values into the
global
 * variables.
 */
void readInputs() {
    uint8_t buf[15];
    uint8_t buflen = sizeof(buf);
    if (rf_driver.recv(buf, &buflen)) {
        //Serial.print(buf[0], BIN);
        //Serial.println(buf[1], BIN);

        Packet pkt;
        decodeStreamByte(buf[0], pkt);
        if (decodeStreamByte(buf[1], pkt)) {
            // Packet is fully received, update variables
            manual_speed    = pkt.speed;
            autopilot_mode = pkt.autopilot;
            headlights_on   = pkt.headlights;
        }
    }
}

/**
 * @brief State machine for the train, uses switch control flow state.
 */
void updateStateMachine() {
    //in_emergency_stop = false; // DISABLE EMERGENCY STOP

    switch (train_state) {
        // Train is emergency stopped
        case EMERGENCY_STOPPED:
            if (!in_emergency_stop) {
                station_trigger_count = 0;
                train_state = autopilot_mode ? AUTO_CRUISING : MANUAL_CONTROL;
            }
            break;

        // Train is stopped
        case STOPPED:
            if (in_emergency_stop)

```

```

        train_state = EMERGENCY_STOPPED;
    else if (millis() - station_dwell_timer >= STATION_DWELL_TIME) {
        station_trigger_count = 0;
        train_state = AUTO_CRUISING;
    }
    break;

// Train is in manual drive mode
case MANUAL_CONTROL:
    if (in_emergency_stop)
        train_state = EMERGENCY_STOPPED;
    else if (autopilot_mode) {
        station_trigger_count = 0;
        train_state = AUTO_CRUISING;
    }
    break;

// Trains autopilot is cruising
case AUTO_CRUISING:
    if (in_emergency_stop)
        train_state = EMERGENCY_STOPPED;
    else if (!autopilot_mode)
        train_state = MANUAL_CONTROL;
    else if (station_trigger_count == 1)
        train_state = AUTO_SLOWING;
    break;

// Train is approaching station
case AUTO_SLOWING:
    if (in_emergency_stop)
        train_state = EMERGENCY_STOPPED;
    else if (station_trigger_count >= 2) {
        station_dwell_timer = millis();
        train_state = STOPPED;
    }
    break;
}
}

/**
 * @brief Writes to outputs, specifically, sets the train target speed, and
 * updates the headlights state.
 */
void applyOutputs() {
    // Set headlight state
    digitalWrite(HEADLIGHT_PIN, headlights_on ? HIGH : LOW);

    // Set indicators

```

```

digitalWrite(AUTOPILOT_INDI, autopilot_mode ? HIGH : LOW);

switch (train_state) {
    // Train needs to be stopped
    case EMERGENCY_STOPPED:
    case STOPPED:
        target_speed = 127;
        break;

    // Train is in manual control
    case MANUAL_CONTROL:
        target_speed = manual_speed;
        break;

    // Trains autopilot is cruising
    case AUTO_CRUISING:
        target_speed = SPEED_CRUISE;
        break;

    // Trains autopilot is slowing down
    case AUTO_SLOWING:
        target_speed = SPEED_SLOW;
        break;
}

// Set the motor state using a linear ramp up
setMotor(linearSpeedRamp(target_speed));
}

//
===== //
// Helper
Functions //
//
===== //

/**
 * @brief Decodes the first and second byte from the serial stream, and puts
 * them into a packet, passed by reference.
 * @param byte_in The byte from the serial read.
 * @param out The reference to the packet currently being built.
 * @return bool, true if the packet is done being built, false if not.
 */
bool decodeStreamByte(uint8_t byte_in, Packet &out) {
    // Static variables for the next call
    static uint8_t last_check = 0; // Store an ID, consistency
check

```

```

    static uint8_t speed_h          = 0;          // Hold the top 5 bits of the
speed
    static bool waiting_for_second = false;       // Flag, tells if we are mid-
packet

    uint8_t check = byte_in >> 6;               // Shift to right, isolate
check
    bool frame     = (byte_in >> 5) & 0x01;       // Shift to right, mask the
frame

    // First byte - the setup
    if (frame == 0) {
        speed_h          = byte_in & 0x1F;       // Mask the last 5 bits
        last_check       = check;               // Remember the check ID
        waiting_for_second = true;               // Set the flag
        return false;                             // Packet is not complete yet
    }

    // Second byte - but never saw the first
    if (!waiting_for_second)
        return false;                           // Out of sync, discard

    // Check doesn't match
    if (check != last_check) {
        waiting_for_second = false;               // Lower flag
        return false;                             // Check mismatch
    }

    // Reassemble packet
    uint8_t speed_l = (byte_in >> 2) & 0x07;    // Shift to the right, then
mask

    out.speed      = (speed_h << 3) | speed_l;   // Shift prev to left, then
append
    out.autopilot  = (byte_in >> 1) & 0x01;       // Shift one to right, then
mask
    out.headlights = byte_in & 0x01;             // Mask the last bit

    // Packet is valid
    waiting_for_second = false;
    return true;
}

/**
 * @brief Uses a linear ramp to gradually increase the speed.
 * @param target The target speed 0-255 int.
 * @return int, the speed at which to move the train.
 */

```

```

uint8_t linearSpeedRamp(int target) {
    // Static variables to remember for the next call
    static float current = 0; // The current speed
    static unsigned long last_time = 0; // Store when the previous call was
made

    unsigned long now = millis(); // Get timestamp
    float dt = (now - last_time) / 1000.0; // Get delta time in seconds
    if (dt <= 0) dt = 0.001; // Check if enough time passed (1ms)
    last_time = now; // Update the last time called
variable

    if (current < target) {
        // Target is bigger, accelerate
        current += ACCEL_RATE * dt;
        // Check if current exceded target
        if (current > target)
            current = target;
    } else if (current > target) {
        // Apply emergency break decel rate if needed
        if (in_emergency_stop)
            current -= DECEL_RATE_E * dt;
        else
            // Target is less, decelerate
            current -= DECEL_RATE * dt;

        // Check if current is below target
        if (current < target)
            current = target;
    }

    // Return the clamped current value to 1 byte
    return constrain((uint8_t)current, 0, 255);
}

/**
 * @brief Set the motor speed and direction based on 1 byte input.
 * @param speed The speed, non negative, artificial zero on center.
 */
void setMotor(uint8_t speed) {
    // Define scope wide variables
    uint8_t pwm = 0; // Pulse-Width-Modulation, actual speed to set
    bool forward = false; // Flag for driving forwards
    bool backward = false; // Flag for driving backwards

    // Define thresholds (deadzones) around the center (127)
    int upper_threshold = 127 + DEADZONE;
    int lower_threshold = 127 - DEADZONE;

```

```

// Move forwards
if (speed > upper_threshold) {
    // Map the speed from threshold
    pwm = (uint8_t)map(speed, upper_threshold, 255, 0, 255);
    forward = true;
// Move backwards
} else if (speed < lower_threshold) {
    // Map the speed from threshold
    pwm = (uint8_t)map(lower_threshold - speed, 0, lower_threshold, 0, 255);
    backward = true;
} else {
    // Deadzone: Input is between 117 and 137
    pwm = 0;
    forward = false;
    backward = false;
}

// Write to hardware
digitalWrite(MOTOR_FORWARD, forward ? HIGH : LOW);
digitalWrite(MOTOR_BACKWARD, backward ? HIGH : LOW);
analogWrite(MOTOR_SPEED, pwm);

// Update the train speed variable
if (forward || backward)
    train_speed = pwm;
else
    train_speed = 0;
}

```